Lab 4: Vibe Coding — Prompt History & Development Log

Student: Vishnu Anand M | **USN:** PES1UG24CS537 | **Assigned Repo:** SETAPESU26/45_snake

Personal Repo: viz2906/SE-Lab-PES1UG24CS537 | **Duration:** 60 mins | **Status:** Completed

1. Lab Objective & Workflow

The objective of this laboratory is to employ modern AI-assisted ('Vibe Coding') pair-programming to analyze, fix, and extend a partially working Pygame Snake starter repository. Requirements mandate addressing all tasks listed in the repository README in under 3 to 4 prompt iterations per task, maintaining separate Git commits for each task, and providing before/after video evidence.

2. Task 1: Refine Collision Detection

Problem: The starter code in game/snake.py did not prevent 180° direction reversals. When pressing Left while moving Right, the snake instantly collided into its second body segment, causing unfair and abrupt termination.

Prompt (Iteration 1/4):

"I am working on the Pygame Snake starter in game/snake.py. Right now, tapping the opposite direction key (e.g. Left while moving Right) causes instant self-collision because the snake reverses straight into its neck segment. Please refine collision detection so that 180-degree reversals are safely ignored when length > 1, and buffer direction updates."

Solution & Verification: Added last_move_direction and next_direction buffers in Snake. Direction changes matching (-cur_dx, -cur_dy) are safely rejected. Unit tests confirmed 100% bug resolution.
Commit: 5c648c6 Task 1: Refine collision detection and prevent instant 180-degree self-collision

3. Task 2: Implement Game Over Condition

Problem: The original game simply printed 'Game over! Final score: X' to the terminal console without displaying any Game Over UI, leaving the player with a frozen screen.

Prompt (Iteration 1/4):

"In game/game_engine.py, the game over condition only logs to the terminal. Please implement an in-game Game Over screen that renders a semi-transparent dark backdrop overlay, displays 'GAME OVER' in bold red text, shows the final score and session high score, and gracefully waits for player input instead of abruptly freezing."

Solution & Verification: Created styled modal dialog overlay with semi-transparent alpha blending, displaying Game Over title, final score, and best score. Handled graceful termination on Q/ESC.
Commit: e46a728 Task 2: Implement Game Over screen with score display and input wait

4. Task 3: Add Replay Option with Difficulty Selection

Problem: Players could not restart or replay without killing and relaunching the program. There were no difficulty/speed options available.

Prompt (Iteration 1/4):

"Extend the Game Over screen in game/game_engine.py to support full replay capabilities with three distinct difficulty levels: Easy (6 moves/s), Medium (10 moves/s), and Hard (16 moves/s). Allow restarting via keys 1, 2, 3 or R, resetting all game state cleanly."

Solution & Verification: Defined DIFFICULTIES presets, added a full restart(difficulty) state reset method, and wired numeric hotkeys [1, 2, 3, R, Q] on the Game Over screen.
Commit: 9e239b3 Task 3: Add replay option with difficulty selection (Easy, Medium, Hard)

5. Task 4: Add Sound Feedback

Problem: The starter game had no audio cues for key events such as eating food or colliding.

Prompt (Iteration 1/4):

"Add sound effects for eating food and for game over events. Create a SoundManager class that safely handles pygame.mixer, synthesizes clean 16-bit WAV audio files (eat.wav chime, game_over.wav defeat sweep), and triggers them without external dependencies."

Solution & Verification: Built self-contained 44.1kHz WAV assets and SoundManager with try/except guarding. Sound triggers added to food consumption and collision handlers.
Commit: b8a8cff Task 4: Add sound effects for food consumption and game over events

6. Commit History & Verification Summary

Commit Hash	Task Description	Scope / Files
`114601f`	Initial Snake starter	Base template
`5c648c6`	Task 1: Refine collision detection & 180° guard	`game/snake.py`
`e46a728`	Task 2: Game Over screen with score & input wait	`game/game_engine.py`
`9e239b3`	Task 3: Replay option with Easy/Med/Hard presets	`game/game_engine.py`
`b8a8cff`	Task 4: Sound feedback for eating & game over	`sound_manager.py, sounds/`

7. Deliverables Verification Checklist

[X] Videos (before and after): Recorded 10s gameplay clips in videos/before_gameplay.mp4 and videos/after_gameplay.mp4.
[X] Updated Code: Fully working, modular Snake game in main.py, game/, sounds/.
[X] Chat history exported: Saved as Markdown (chat_history_vibe_coding.md) and PDF (chat_history_vibe_coding.pdf).
[X] Under 3-4 attempts: All four tasks accomplished and verified in 1 prompting iteration each.